

WEEK 04 · REFERENCE

THE GRID CATALOGUE

The lecture is a story. This is the lookup table. Keep it open while you build.

WHAT IS IN HERE

Every track keyword, every placement form, and the alignment matrix in full.

HOW TO READ IT

Left column is what you write. Right column is what the browser does with it.

WHEN TO OPEN IT

Mid-build, when a track list does what you did not ask for.

TRACK SIZING

VALUE	MEANS	REACH FOR IT WHEN
<code>200px</code>	exactly that	the thing has a real size — an icon, a control
<code>25%</code>	a share of the container, ignoring <code>gap</code>	almost never in a grid. Use <code>fr</code>
<code>1fr</code>	a share of what is <i>left over</i>	the default answer for a flexible column
<code>auto</code>	content size, then stretches into spare room	a label column beside a field
<code>min-content</code>	as narrow as possible without overflowing	a column that should hug its longest word
<code>max-content</code>	as wide as it wants, wrapping nothing	rarely, and never for prose
<code>minmax(a, b)</code>	a floor and a ceiling	whenever a track has both a limit and freedom
<code>fit-content(a)</code>	content size, capped at <code>a</code>	a caption column that must not run away

REPEAT, AUTO-FIT AND AUTO-FILL

WRITTEN

```
repeat(3, 1fr)
```

```
repeat(2, 1fr 140px)
```

```
repeat(auto-fit, minmax(15rem, 1fr))
```

```
repeat(auto-fill, minmax(15rem, 1fr))
```

PRODUCES

three equal columns, always three

four tracks — `repeat()` takes a **pattern**

as many as fit; **empty tracks collapse** and the items grow

as many as fit; **empty tracks stay** at their size

THEY ARE IDENTICAL...

...whenever there are enough items to fill the row. That is why so many people believe they are the same keyword.

CHOOSING

`auto-fit` for a card row. `auto-fill` when an empty slot is information — a calendar, a seating chart.

PLACEMENT

WRITTEN	MEANS
<code>grid-column: 1 / 3</code>	from line 1 to line 3 — that is two columns
<code>grid-column: 1 / span 2</code>	the same thing, counted as a width
<code>grid-column: 1 / -1</code>	all the way across, whatever the track count
<code>grid-row: 2 / 4</code>	from row line 2 to row line 4
<code>grid-area: 2 / 1 / 4 / 3</code>	row-start / column-start / row-end / column-end
<code>grid-area: main</code>	the area called main in the container's picture

► **THREE TRACKS MEANS FOUR LINES.** You count the edges, not the things.

NAMED AREAS

on the container

`grid-template-areas`, one quoted string per row.

on each item

`grid-area: <name>` and nothing else.

a dot

`.` is a deliberately empty cell.

rule one

Every row needs the **same number of cells**.

rule two

Every named area must be a **rectangle**. There is no L shape.

when you break either

The declaration is dropped in silence and you get auto-placement.

```
.page {  
  grid-template-areas:  
    "head head head"  
    "side main aside"  
    "foot foot foot";  
}  
  
.masthead { grid-area: head; }  
.sidebar { grid-area: side; }
```

THE IMPLICIT GRID

PROPERTY	DOES
<code>grid-auto-rows</code>	sizes rows nobody declared
<code>grid-auto-columns</code>	the same for columns
<code>grid-auto-flow: row</code>	the default — fill along rows
<code>grid-auto-flow: column</code>	fill along columns instead
<code>grid-auto-flow: dense</code>	let a later, smaller item back-fill a hole

THE COST OF `dense`

It changes the **visual** order without touching the DOM, so what people see and what they tab through stop matching. Fine for independent cards. A bug in a form.

WHY IMPLICIT ROWS MATTER

A list from data has an unknown length. The implicit grid is what makes that work without counting anything.

ALIGNMENT — THE WHOLE MATRIX

PROPERTY	ALIGNS	ALONG
<code>justify-content</code>	the tracks inside the container	inline (horizontal)
<code>align-content</code>	the tracks inside the container	block (vertical)
<code>justify-items</code>	every item inside its cell	inline
<code>align-items</code>	every item inside its cell	block
<code>justify-self</code>	one item inside its cell	inline
<code>align-self</code>	one item inside its cell	block
<code>place-items</code>	<code>align-items</code> then <code>justify-items</code>	
<code>place-content</code>	<code>align-content</code> then <code>justify-content</code>	

► **UNLIKE FLEXBOX** — these axes never swap. `justify-` is always inline, whatever else you change.

GRID OR FLEX

YOU ARE BUILDING	USE	BECAUSE
a page shell	Grid	rows and columns must line up together
a card row of unknown length	Grid	<code>auto-fit</code> removes the media query
a dashboard, a gallery	Grid	two dimensions, and items that span
a navbar	Flex	one axis, sizes from content
a row of buttons	Flex	one axis. Grid here is the classic overreach
the inside of a card	Flex	a column, with one child taking the slack
a chip with an icon	Flex	two things, centred, on one axis
a form with a label column	Grid	the labels must align across rows

You don't have to choose one over the other — Flexbox can be used inside a grid cell.

RESPONSIVE — THE UNITS

UNIT	RELATIVE TO	USE FOR
<code>rem</code>	the root font size — the reader's own setting	type, spacing, breakpoints
<code>em</code>	the element's own font size	a media query, so it moves with the reader
<code>ch</code>	the width of a "0"	<code>max-width</code> on prose
<code>vw</code>	1% of the viewport width	inside <code>clamp()</code> , never alone
<code>dvh</code>	the viewport height <i>as it is now</i>	full-height layouts on a phone
<code>vh</code>	the largest viewport height	rarely — the address bar will cover the bottom

clamp() , AND THE ACCESSIBILITY TRAP

RIGHT

```
h1 {  
  font-size: clamp(1.6rem, 1rem + 2.2vw, 2.6rem);  
}
```

The `rem` term keeps the reader's font setting in the answer.

WRONG

```
h1 {  
  font-size: clamp(1.6rem, 4.2vw, 2.6rem);  
}
```

Pure `vw`. Somebody who set a larger default font is ignored entirely.

► **CLAMP(MIN, PREFERRED, MAX)** returns the middle one unless it falls outside the other two.



IMAGES

TOOL	JOB
<code>aspect-ratio</code>	decides how big the box is, and reserves it before the file arrives
<code>width / height</code> attributes	the same reservation, for free, before the CSS loads
<code>object-fit: cover</code>	fills the box and crops. For photographs
<code>object-fit: contain</code>	fits inside and leaves space. For logos and diagrams
<code>srcset="... 800w"</code>	the candidate files, each labelled with its real width
<code>sizes="(min-width: 60em) 33vw, 100vw"</code>	how wide the slot will be, so the browser can choose before layout

► **TWO PROPERTIES, TWO JOBS.** `aspect-ratio` sizes the box; `object-fit` handles the picture inside it.

MOTION

CHEAP

`transform · opacity · colour ·
box-shadow`

The compositor moves an existing layer. No layout, no paint.

EXPENSIVE

`width · height · top · left ·
margin · padding · font-size`

Layout and paint again, every frame.

TRANSITION: ALL

Covers whatever happens to change — including the expensive ones. Nobody who writes it knows what it covered.

```
@media (prefers-reduced-motion: reduce) {  
  :root { --duration: 0ms; }  
}
```

THE FIVE MISTAKES

THE MISTAKE

Grid inside a button row

`auto-fit` and `auto-fill` are the same

Desktop-first breakpoints

Fixed heights on layout regions

Animating `width` or `left`

WHAT IS TRUE

One axis is Flexbox. Grid is for the page

Identical only while there are enough items. With fewer, fit collapses and fill does not

`min-width` accumulates; `max-width` has to undo itself

A region is as tall as its content. Fixed heights break in another language

Layout and paint every frame. Use `transform`

► **AND THE SIXTH** — `1fr` is `minmax(auto, 1fr)`, so it will not shrink below its content. `minmax(0, 1fr)`.

WHERE TO LOOK IT UP

IN THIS REPO

`demos/02-tracks/` — type a track list, see what the browser did.

`demos/04-auto-fit-fill/` — the one everybody gets wrong.

IN THE BROWSER

DevTools, select a container, click the `grid` badge. It draws the lines, the numbers and the gaps.

ELSEWHERE

MDN's CSS grid layout guide.
And CSS Grid Garden — twenty minutes, and it sticks.